Lesson notes · for you

Batch and stream

Data engineering · Level 4 · Lesson 4

Time
15 min

Materials
Your phone or a computer, for the Python step; Paper, for the five systems

Learning goals

- Explain batch and stream processing and their trade-offs.
- Tame noise with a moving window, and see the delay it adds.
- Choose batch, stream or both for real systems.

Activity: Human stream

1. Readings (3 min): One person reads out river levels one at a time, including a splash.
2. Window 1 (3 min): Shout “alarm” whenever a single reading is above 2.0. How many false alarms?
3. Window 3 (5 min): Now alarm only when the average of the last three is above 2.0. Fewer false alarms, but when does the real alert come?
4. Decide (4 min): List five systems and decide for each, with the cost of a late and of a false answer.

Use the readings from the Python step, so the class can check its alerts against the code.

Discussion

- Why is a batch easy to rerun?
- What did the bigger window cost us?
- Why use the time an event happened, not when it arrived?

Check yourself

- Name one system that needs a stream and one that a batch serves better, with a reason each.

Try it this week
List five systems you use, like bus times, exam results and a bank alert. For each one, decide: batch or stream, and why?

Worksheet

Batch and stream

Data engineering · Level 4 · Lesson 4

Name _____ Class _____ Date _____

1 **You make the averaging window bigger. What happens?**

◯ Nothing changes ◯ More false alarms, and earlier alerts ◯ Fewer false alarms, but later alerts

2 **Match each system to the processing it needs.**

A flood warning	• • • •	A nightly batch: cheap and easy to rerun
A monthly attendance report	• • • •	A stream: minutes matter
A card fraud alert	• • • •	A batch: once, carefully checked
The year's exam results	• • • •	A stream: stop it before the next payment

3 **A moving-average alert. first_alert(readings, k) should return the first minute where the average of the last k readings (including that minute's) is above 2.0 m. The window slice is off by one: fix it. With k = 1 the splash at minute 2 alarms; with k = 3 the alert comes at minute 8.**

```
readings = [
    1.2, 1.3, 2.6, 1.3, 1.4, 1.6,
    1.9, 2.1, 2.3, 2.4, 2.6,
]
def first_alert(readings, k):
    for i in range(k - 1, len(readings)):
        window = readings[i - k:i]
        if sum(window) / k > 2.0:
            return i
    return None
```

Write your program:

4 **Three calm readings and one splash. What does this program print?**

```
r = [1.0, 1.0, 4.0, 1.0]
print(sum(r[-3:]) / 3)
```

◯ 1.75 ◯ 2.0 ◯ 4.0

5 **How should a stream cope with messy events? Choose all that apply.**

☐ Make processing idempotent, because events can arrive twice ☐ Alert when a sensor goes quiet

☐ Use the time the event happened, not when it arrived ☐ Drop any event that arrives late

✎ **Pick an app on your phone that tells you something "live" (a bus, a delivery, a score). What would it be like as a nightly batch? What would it cost to make it faster?**

Check your answers

Batch and stream

Data engineering · Level 4 · Lesson 4

1 **Fewer false alarms, but later alerts**

More readings in the window smooth out splashes, but a real rise takes longer to lift the average above the line.

2 **A flood warning — A stream: minutes matter**
A monthly attendance report — A nightly batch: cheap and easy to rerun
A card fraud alert — A stream: stop it before the next payment
The year's exam results — A batch: once, carefully checked

Ask what a late answer costs. Floods and fraud can't wait, so they stream; reports and results can, so a batch gives them more care for less money.

3

```
readings = [
    1.2, 1.3, 2.6, 1.3, 1.4, 1.6,
    1.9, 2.1, 2.3, 2.4, 2.6,
]
def first_alert(readings, k):
    for i in range(k - 1, len(readings)):
        window = readings[i - k + 1:i + 1]
        if sum(window) / k > 2.0:
            return i
    return None
```

A slice stops before its end index, so readings[i - k + 1:i + 1] is exactly the k readings ending at minute i.

4 **2.0**

r[-3:] is the last three readings, 1.0, 4.0 and 1.0, and their average is 2.0.

5 **Use the time the event happened, not when it arrived; Make processing idempotent, because events can arrive twice; Alert when a sensor goes quiet**

Late events still happened, so place them by their event time instead of dropping them.